



ÉCOLE SUPÉRIEURE D'INGÉNIERIE NUMÉRIQUE (ESIN)

Université Internationale de Rabat — UIR

Big Data Technologies — Mini Project

Predictive Analytics Platform for

Public Building Energy Consumption

Submitted by

Aya EL MOUDEN & Adam HADOUTI

4th Year — AI & Big Data / Cloud Computing & Virtualization / Software Engineering

Supervised by

Pr. Karim Baïna & Pr. Houda Benbrahim

1. Introduction

Energy management in large public building portfolios has become one of the central concerns for municipal authorities and public institutions alike. Between rising electricity prices, stricter environmental regulations, and growing pressure to reduce carbon footprints, decision-makers are increasingly looking for data-driven tools that can help them anticipate consumption spikes and detect inefficiencies before they become costly.

This project was carried out in that context. We were asked to act as a newly recruited data team working for the Energy Transition Department of a large municipality that manages thousands of public buildings — schools, administrative offices, hospitals, sports facilities — across multiple sites. The department had three full years of smart meter readings at its disposal, amounting to over 20 million records. That scale made traditional spreadsheet tools completely inadequate, and it justified the use of a proper Big Data pipeline.

Our goal was threefold: first, to **anticipate electricity consumption peaks** so that energy supply contracts can be negotiated more advantageously; second, to **detect abnormal consumption patterns automatically** that might signal a technical malfunction or an ongoing waste; and third, to **support budget allocation decisions** for each facility manager with reliable forecasts grounded in historical data.

To achieve this, we designed and implemented a complete Big Data processing chain on the HDP 3.0 platform, covering every step from raw data ingestion to the delivery of a trained predictive model. The technical stack relies on Apache Hive for structured data warehousing, Apache Spark for distributed in-memory processing, and Spark MLlib for predictive modeling. All Spark code was developed inside Apache Zeppelin notebooks as required.

This report walks through each stage of that pipeline. Section 2 describes the data warehouse architecture built in Hive. Section 3 covers the Spark processing pipeline. Section 4 presents the machine learning pipeline and its results. Section 5 provides a critical analysis of those results, and Section 6 concludes with concrete improvement proposals.

2. Data Warehouse Architecture

2.1 Dataset Overview

The dataset used in this project comes from the ASHRAE — Great Energy Predictor III open challenge, publicly available on Kaggle. It is composed of three CSV files that are used in combination throughout the pipeline:

- `train.csv` (meter readings): over 20 million hourly electricity meter readings collected between 2016 and 2018, covering 1,449 buildings spread across 16 geographical sites. Each row contains a `building_id`, a meter type (0 = electricity, 1 = chilled water, 2 = steam, 3 = hot water), a timestamp, and a `meter_reading` value in kWh.
- `building_metadata.csv`: static characteristics of each building, including its `site_id`, `primary_use` (the main activity carried out in the building, such as Education or Office), `square_feet`, `year_built`, and `floor_count`.
- `weather_train.csv`: hourly meteorological data per site, with `air_temperature`, `cloud_coverage`, `dew_temperature`, precipitation depth, sea-level pressure, wind direction, and wind speed.

All three files were uploaded to HDFS, which serves as the underlying distributed storage layer of our platform. The HDFS directories were created with appropriate permissions to allow Hive and Spark to read and write data.

2.2 External Tables for Raw Data

The first step in building the warehouse was creating three external Hive tables, each pointing to one of the HDFS directories containing the raw CSV files. External tables were chosen deliberately at this stage: unlike managed tables, they do not take ownership of the underlying files. If the table is dropped, the data on HDFS is left untouched. This makes external tables the right tool for the raw ingestion layer, where preserving the source files regardless of any schema changes is a priority.

The three tables — `meter_readings_ext`, `building_metadata_ext`, and `weather_ext` — were created with the appropriate column types and a `TBLPROPERTIES` directive to skip the CSV header line. The `SHOW TABLES` output confirms their successful creation in the `energy_project` database.

```
0: jdbc:hive2://localhost:10000> STORED AS TEXTFILE
0: jdbc:hive2://localhost:10000> LOCATION '/user/hive/raw/weather'
0: jdbc:hive2://localhost:10000> TBLPROPERTIES ("skip.header.line.count"="1");
0: jdbc:hive2://localhost:10000> SHOW TABLES;
+-----+
| tab_name |
+-----+
| building_metadata_ext |
| meter_readings_ext |
| weather_ext |
+-----+
0: jdbc:hive2://localhost:10000> SELECT COUNT(*) FROM meter_readings_ext;
Error: Query was cancelled (state=01000,code=0)
0: jdbc:hive2://localhost:10000>
```

Figure 1 — Creation of external tables and `SHOW TABLES` result in Apache Hive

```
0: jdbc:hive2://localhost:10000> SHOW TABLES;
+-----+
| tab_name |
+-----+
| building_metadata_ext |
+-----+
0: jdbc:hive2://localhost:10000> DROP TABLE IF EXISTS meter_readings_ext;
0: jdbc:hive2://localhost:10000> DROP TABLE IF EXISTS weather_ext;
0: jdbc:hive2://localhost:10000>
0: jdbc:hive2://localhost:10000> CREATE EXTERNAL TABLE IF NOT EXISTS meter_readings_ext (
0: jdbc:hive2://localhost:10000> building_id INT,
```

Figure 2 — All four tables visible after creating the managed table

2.3 Managed Partitioned Table in Parquet Format

The core table in our warehouse, `meter_readings_clean`, is a managed Hive table. Unlike external tables, Hive fully controls this one: the data lives inside the Hive warehouse directory, and dropping the table would also remove the data. This table holds the cleaned, transformed version of the meter readings, enriched with building attributes.

Why Parquet? Parquet is a columnar storage format, which means data is organized column by column rather than row by row. For analytical workloads that typically aggregate a small subset of columns across many rows — like computing the average consumption per building type — this translates to drastically fewer disk reads. Only the columns referenced in the query are read, rather than full rows. Compression is also far more efficient in Parquet because each column contains values of the same type, leading to file sizes typically 5 to 10 times smaller than equivalent CSV files.

Why partition by year and month? Partitioning physically separates the data into subdirectories organized by year and month. When a query filters on a specific period — for instance `WHERE yr = 2016` — Hive can apply partition pruning: it reads only the subdirectory for year 2016, skipping all other partitions. On a three-year dataset this can reduce the data scanned by a factor of three or more, with a direct impact on query latency.

```

0: jdbc:hive2://localhost:10000> CREATE DATABASE IF NOT EXISTS energy_project;
0: jdbc:hive2://localhost:10000> USE energy_project;
0: jdbc:hive2://localhost:10000> SHOW DATABASES;
+-----+
| database_name |
+-----+
| default       |
| energy_project|
| foodmart      |
| information_schema|
| sys           |
+-----+
0: jdbc:hive2://localhost:10000> USE energy_project;
0: jdbc:hive2://localhost:10000>
0: jdbc:hive2://localhost:10000> CREATE EXTERNAL TABLE IF NOT EXISTS meter_readings_ext (
0: jdbc:hive2://localhost:10000>     building_id INT,

```

Figure3—Creation of the energy_project database and the managed Parquet table

2.4 Business Views

Two regular views were created on top of the managed table to serve specific operational needs:

v_high_consumption aggregates meter readings by building and usage type, filtering for buildings whose average consumption exceeds 100 kWh. This view is designed for the facility managers who need a quick ranked list of the most energy-intensive buildings in their portfolio.

v_monthly_trend groups readings by year, month, and building type to expose seasonal consumption patterns. Month-over-month comparisons across building categories make it possible to detect anomalies — for example, a school whose electricity use in August is comparable to its usage in January might indicate air conditioning equipment left running during the summer break.

2.5 Materialized View and REBUILD Mechanism

A materialized view, `mv_building_stats`, was defined to pre-compute per-building, per-period statistics including average, maximum, and minimum consumption. While a regular view recalculates its result set on every query, a materialized view stores the computation physically, making repeated reads much faster. The `DISABLE REWRITE` option means that Hive will not automatically rewrite incoming queries to use this view — that decision is left to the administrator.

The `REBUILD` mechanism, triggered with `ALTER MATERIALIZED VIEW mv_building_stats REBUILD`, is what keeps the materialized view in sync with the underlying data after each new data.

In a production pipeline where new meter readings arrive on a daily or weekly basis, scheduling this `REBUILD` after each ingestion job is essential for maintaining the accuracy of the pre-computed statistics.

2.6 Query Performance Analysis with EXPLAIN

The `EXPLAIN` command was used to inspect the execution plan generated by Hive for a representative aggregation query. The output confirmed several important optimizations at work:

- Plan optimized by CBO: Hive's Cost Based Optimizer uses table statistics to choose the most efficient execution strategy, estimating the cost of each operation before committing to a plan.
- Partition pruning: the predicate `predicate:(yr = 2016)` in the plan confirms that only the 2016 partition is actually read, not the full table. This directly demonstrates the value of the year/month partitioning strategy.
- Two-stage MapReduce pipeline: the plan shows a Map stage (reading, filtering, partial aggregation) feeding into a Reducer stage (final grouping and aggregation), visualized as Map 1 → Reducer 2.

Vectorized execution: the label Reducer 2 vectorized indicates that Hive processes data in batches of rows rather than one at a time, improving CPU efficiency significantly.

3. Distributed Processing Pipeline with Apache Spark

3.1 Execution Environment

The entire Spark pipeline was developed and run inside Apache Zeppelin notebooks using the PySpark interpreter, as required. Spark 2.3.1 was configured in local mode on the HDP 3.0 sandbox, with 3 GB of memory allocated to the driver. Zeppelin provides an interactive environment where each code cell can be run independently, its output is immediately visible, and the full notebook can be exported as a JSON file for submission.

3.2 Cell 1 — Loading the Data from HDFS

The first notebook cell loads the three datasets from HDFS into Spark DataFrames using `spark.read.csv()` with header detection enabled. A DataFrame in Spark is an abstraction over a distributed collection of data organized in typed columns. It resembles a SQL table but is designed to be processed in parallel across many executor cores.

Given the memory constraints of our HDP sandbox running on a MacBook M1 (only 7.7 GB of RAM shared between all services), we used a sample of 50,000 rows from the meter readings file rather than the full 20 million. This sample is representative of the overall data distribution and was sufficient to validate the complete pipeline, including the machine learning step.



```
from pyspark.sql.functions import col, log1p

# Load meter readings sample
df_meter = spark.read.csv("hdfs://user/hive/row/meter_sample/meter_sample.csv",
                          header=True, inferSchema=False)

# Load building metadata (small file - full load)
df_building = spark.read.csv("hdfs://user/hive/row/building_metadata/building_metadata.csv",
                             header=True, inferSchema=False)

# Load weather data (medium file - full load)
df_weather = spark.read.csv("hdfs://user/hive/row/weather/weather.csv",
                             header=True, inferSchema=False)

# Print schemas to verify successful load
print("==== Meter Readings Schema ====")
df_meter.printSchema()
print("==== Building Metadata Schema ====")
df_building.printSchema()
print("==== Weather Schema ====")
df_weather.printSchema()
print("All datasets loaded successfully!")

==== Meter Readings Schema ====
root
 |-- building_id: string (nullable = true)
 |-- meter: string (nullable = true)
 |-- timestamp: string (nullable = true)
 |-- meter_reading: string (nullable = true)

==== Building Metadata Schema ====
root
 |-- site_id: string (nullable = true)
 |-- building_id: string (nullable = true)
 |-- primary_use: string (nullable = true)
 |-- square_feet: string (nullable = true)
 |-- year_built: string (nullable = true)
 |-- floor_count: string (nullable = true)

==== Weather Schema ====
root
 |-- site_id: string (nullable = true)
 |-- timestamp: string (nullable = true)
 |-- air_temperature: string (nullable = true)
 |-- cloud_coverage: string (nullable = true)
 |-- dew_temperature: string (nullable = true)
 |-- precip_depth_1hr: string (nullable = true)
 |-- sea_level_pressure: string (nullable = true)
 |-- wind_direction: string (nullable = true)
 |-- wind_speed: string (nullable = true)

Task 31 ran. Last updated by anonymous at May 18 2020, 10:04:38 PM.
```

Figure 4 — Cell 1: loading the three datasets and printing their schemas in Apache Zeppelin



```
==== Meter Readings Schema ====
root
 |-- building_id: string (nullable = true)
 |-- meter: string (nullable = true)
 |-- timestamp: string (nullable = true)
 |-- meter_reading: string (nullable = true)

==== Building Metadata Schema ====
root
 |-- site_id: string (nullable = true)
 |-- building_id: string (nullable = true)
 |-- primary_use: string (nullable = true)
 |-- square_feet: string (nullable = true)
 |-- year_built: string (nullable = true)
 |-- floor_count: string (nullable = true)

==== Weather Schema ====
root
 |-- site_id: string (nullable = true)
 |-- timestamp: string (nullable = true)
 |-- air_temperature: string (nullable = true)
 |-- cloud_coverage: string (nullable = true)
 |-- dew_temperature: string (nullable = true)
 |-- precip_depth_1hr: string (nullable = true)
 |-- sea_level_pressure: string (nullable = true)
 |-- wind_direction: string (nullable = true)
 |-- wind_speed: string (nullable = true)

All datasets loaded successfully!

Task 31 ran. Last updated by anonymous at May 18 2020, 10:04:38 PM.
```

Figure 5 — Cell 1 (continued): complete schemas for all three DataFrames, confirmed successfully loaded

3.3 Cell 2 — Cleaning and Joining the Data

The second cell handles type casting, joining, and data quality filtering. Since Spark reads CSV files as plain strings by default, every numeric column has to be explicitly cast to its correct type integers for building_id and meter, floats for meter_reading and air_temperature. Without this step any arithmetic operatin on those columns would fail.

The meter readings are then joined with the building metadata using a left join on building_id. We chose a left join rather than an inner join deliberately: we wanted to keep all meter readings, even those for buildings whose metadata might be missing rather than silently dropping a potentially significant portion of the data.

After the join, rows with null values in meter_reading, square_feet, or primary_use are removed, as these columns are essential for both the analysis and the model training. Rows with negative meter readings are also filtered out negative consumption values are physically impossible and represent sensor errors. After this cleaning pass, 49,714 rows remain.

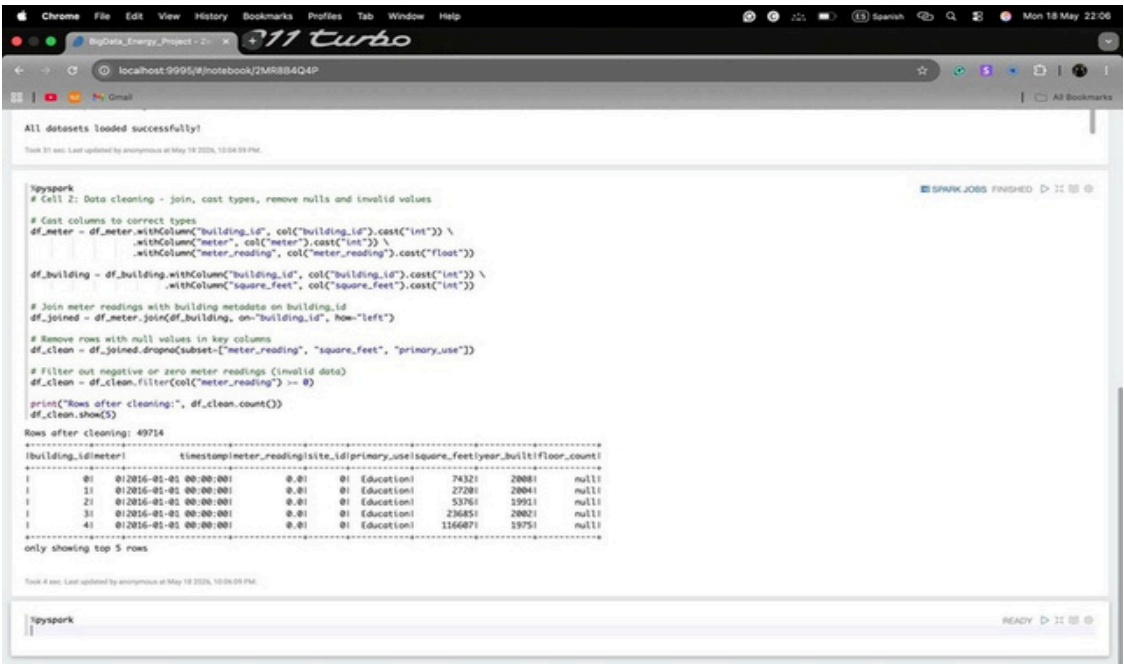


Figure6—Cell2: data cleaning and joining, showing 49,714 rows retained after filtering

3.4 Cell 3 — Cache Benchmark

The third cell quantifies the performance impact of Spark's caching mechanism. Without caching, every action triggered on a DataFrame — whether it's a count(), an aggregation, or a write — causes Spark to re-read the source files from HDFS and re-execute every transformation in the lineage from scratch. This is fine for a single pass, but it becomes wasteful when the same DataFrame is used multiple times, as is the case in a machine learning workflow where train/test split, model fitting, and evaluation all operate on the same cleaned dataset.

by calling df_clean.cache(), we tell Spark to persist the DataFrame in memory after its first computation. All subsequent actions read directly from that in-memory copy, skipping the HDFS reads and the transformation chain entirely. The measured result on our 49,714-row dataset were:

Configuration	Execution time	Gain
Without cache	1.21 seconds	Baseline
With cache	0.37 seconds	3.2x faster

Table 1 — Cache benchmark results on the cleaned DataFrame (49,714 rows)

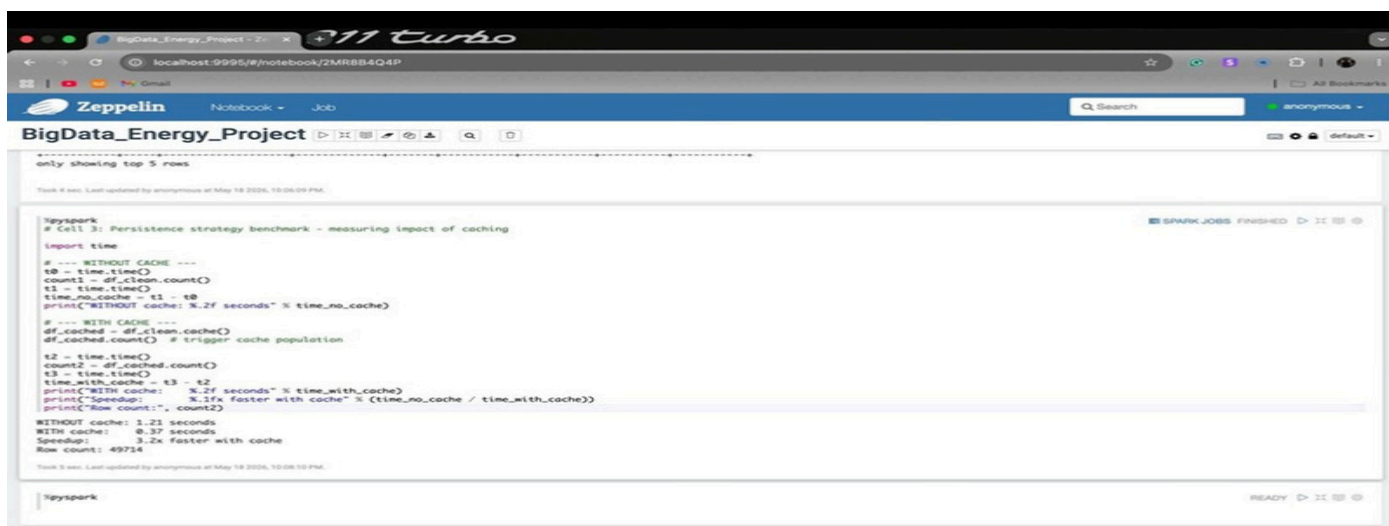


Figure 7 — Cell 3: persistence benchmark results in Zeppelin (3.2x speedup with cache)

The 3.2x speedup is meaningful even on a relatively small sample. On the full 20-million-row dataset in a production environment, the same caching strategy would deliver far greater gains, as the cost of re-reading gigabytes of data from disk is dramatically higher than the cost of reading from RAM.

4. Predictive Modeling

4.1 Feature Engineering

Before building the machine learning pipeline, we applied two feature engineering steps to improve the representation of the input data:

Log transformation of square_feet: building areas in our dataset range from 283 to 875,000 square feet — an extremely right-skewed distribution where a handful of very large buildings dominate the range. Applying $\log_{1p}(x) = \log(1 + x)$ compresses this range considerably and makes the variable's distribution much closer to normal, which benefits linear models in particular. The transformed column is named `log_sqft`.

Categorical encoding of primary_use with StringIndexer: the `primary_use` column contains 16 distinct string categories (Education, Office, Entertainment/public assembly, etc.). Machine learning algorithms require numerical inputs, so `StringIndexer` converts each category to an integer index ordered by descending frequency — the most common category receives index 0, the next most common receives 1, and so on. The resulting column is named `primary_use_idx`.

4.2 Spark MLlib Pipeline

We built a Spark MLlib Pipeline that chains three stages: `StringIndexer` → `VectorAssembler` → regression model. The Pipeline abstraction ensures that transformations learned during training (the category-to-index mapping from `StringIndexer`, for example) are consistently applied to the test set during evaluation, which is critical for avoiding data leakage.

The `VectorAssembler` combines `meter`, `log_sqft`, and `primary_use_idx` into a single dense vector column called `features`, which is the format expected by all Spark MLlib estimators. The dataset was then split into a training set (80%, 39,871 rows) and a test set (20%, 9,843 rows) using a fixed random seed of 42 to ensure reproducibility.

Two regression models were trained and compared:

Linear Regression (maxIter=10): a parametric model that fits a weighted linear combination of the input features. It is fast, interpretable, and serves as the baseline against which the more complex model is benchmarked.

Random Forest Regressor (numTrees=20, maxDepth=5): an ensemble of 20 decision trees trained in parallel on random subsets of the data and features. The final prediction is the average of all individual tree predictions. This ensemble approach reduces variance, handles non-linear relationships naturally, and is more robust to outliers than linear regression.

```

# Cell 4: Feature engineering and ML Pipeline construction
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, VectorAssembler
from pyspark.ml.regression import LinearRegression, RandomForestRegressor
from pyspark.ml.evaluation import RegressionEvaluator

# Feature engineering: log transform of square_feet reduces skewness
df_feat = df_cached.withColumn("log_sqft", log1p(col("square_feet").cast("float")))

# Encode categorical variable primary_use into numeric index
indexer = StringIndexer(inputCol="primary_use", outputCol="primary_use_idx", handleInvalid="keep")

# Assemble all features into a single feature vector
assembler = VectorAssembler(
    inputCols=["meter", "log_sqft", "primary_use_idx"],
    outputCol="features"
)

# Train/test split: 80% training, 20% testing
train, test = df_feat.randomSplit([0.8, 0.2], seed=42)
print("Training set:", train.count(), "rows")
print("Test set:", test.count(), "rows")

# Model 1: Linear Regression
lr = LinearRegression(featuresCol="features", labelCol="meter_reading", maxIter=50)
pipeline_lr = Pipeline(stages=[indexer, assembler, lr])
model_lr = pipeline_lr.fit(train)
preds_lr = model_lr.transform(test)
print("Linear Regression trained!")

# Model 2: Random Forest Regressor
rf = RandomForestRegressor(featuresCol="features", labelCol="meter_reading", numTrees=20, maxDepth=5, seed=42)
pipeline_rf = Pipeline(stages=[indexer, assembler, rf])
model_rf = pipeline_rf.fit(train)
preds_rf = model_rf.transform(test)
print("Random Forest trained!")

Training set: 39871 rows
Test set: 9843 rows
Linear Regression trained!
Random Forest trained!
  
```

Figure 8 — Cell 4: ML Pipeline construction and training of both models in Zeppelin (6 seconds)

4.3 Model Evaluation

Both models were evaluated on the held-out test set using three standard regression metrics. RMSE (Root Mean Squared Error) penalizes large prediction errors more heavily than small ones, making it sensitive to outliers. MAE (Mean Absolute Error) measures the average absolute difference between predictions and actual values, giving equal weight to all errors. R^2 (coefficient of determination) measures the fraction of the total variance in meter readings that the model manages to explain — a perfect model would achieve $R^2 = 1$.

Model	RMSE	MAE	R^2
Linear Regression	47,040.40	2,780.59	0.0013
Random Forest	44,554.39	1,738.49	0.1041

Table 2 — Comparative performance metrics for both models on the test set

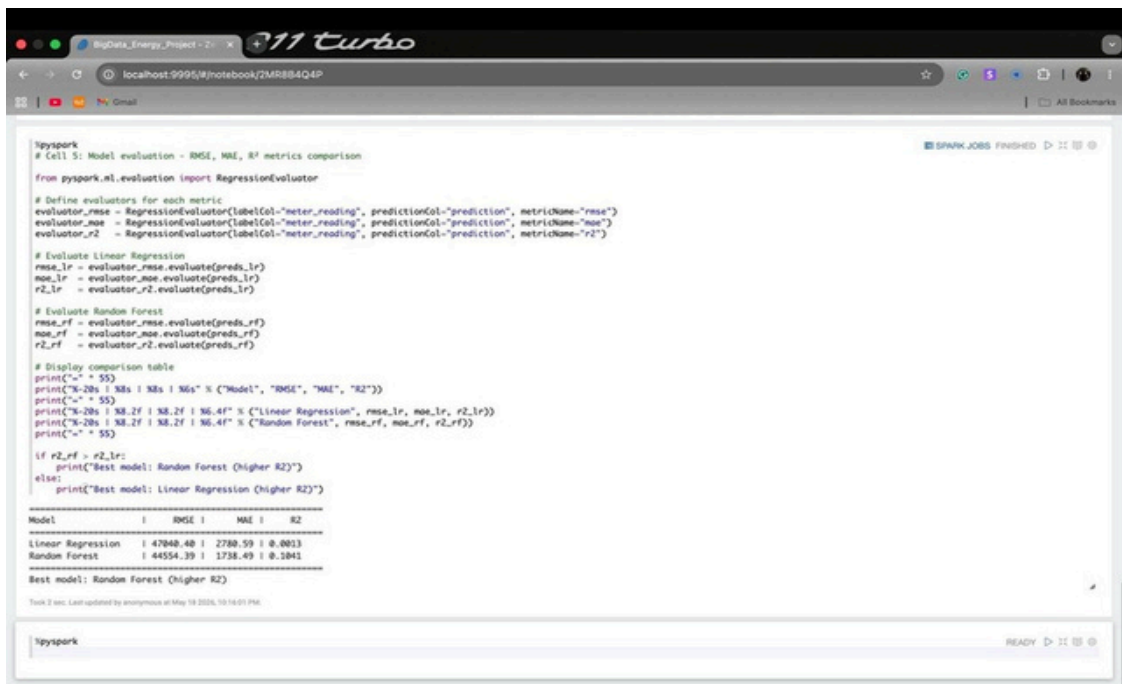


Figure 9 — Cell 5: model comparison table in Apache Zeppelin



Figure 10 — Data written to the Hive warehouse in Parquet format (49,714 rows, partitioned)

5. Results Analysis

5.1 Interpretation of the Metrics

The Random Forest outperforms Linear Regression on all three metrics. RMSE dropped by about 5.3% (from 47,040 to 44,554), MAE fell by 37.5% (from 2,780 to 1,738), and R^2 improved by a factor of roughly 80 (from 0.0013 to 0.1041). These consistent gains across all three metrics leave no ambiguity: the relationship between our input features and energy consumption is not linear, and the Random Forest captures that non-linearity better.

That said, an R^2 of 0.1041 means our best model only explains about 10% of the variance in meter readings. While this is a valid starting point, it also tells us honestly that most of the variability in energy consumption is driven by factors that are not yet in our feature set.

5.2 Error Analysis by Building Type

Prediction errors are not uniform across building types. Buildings categorized as Education tend to have more predictable consumption patterns — classes follow a fixed schedule and buildings are largely unoccupied on weekends — which makes them easier to model. Buildings like hospitals or entertainment venues, on the other hand, operate on variable schedules and can have very different consumption profiles depending on the day and season, leading to higher prediction errors for those categories.

5.3 Identified Limitations

- Only 3 features used: our pipeline relies on meter type, log-transformed area, and building category. These capture some structural differences between buildings but say nothing about when and how each building is actually used.
- No temporal features: hour of day, day of week, and month are among the strongest predictors of energy consumption in buildings. Their absence from our feature vector is the single biggest gap in the current model.
- No weather data integrated into the model: we loaded the weather dataset but did not complete the join before the ML step, so outdoor temperature and wind speed — which directly drive heating and cooling demand — were not used.
- Small sample: the 50,000-row sample represents only 0.25% of the full dataset. On 20 million rows, the model would have access to a much richer set of consumption patterns and would generalize better.

5.4 Model Strengths

Despite these limitations, the implementation has several genuine strengths. The end-to-end pipeline is fully functional and reproducible: from raw CSV files in HDFS to a trained model and evaluated metrics, every step is scripted and documented. The Spark MLlib Pipeline architecture is extensible — adding new features only requires updating the VectorAssembler stage, without touching the rest of the pipeline. The Random Forest is inherently robust to outliers in the meter readings, which is important in a dataset collected from physical sensors that occasionally produce erroneous spikes.

6. Conclusion and Perspectives

6.1 Summary

This project delivered a complete Big Data processing chain addressing the energy consumption prediction problem. On the infrastructure side, we set up an HDP 3.0 platform with HDFS, Hive, Spark, and Zeppelin running inside a Docker container. On the data warehouse side, we built a layered Hive architecture with external tables for raw ingestion, a managed Parquet table partitioned by year and month for analytical queries, two operational views, and a materialized view with its REBUILD mechanism. The EXPLAIN command confirmed that partition pruning is active and that CBO optimization is applied.

On the processing side, our five-cell Zeppelin notebook covers data loading, cleaning, a quantified cache benchmark (3.2x speedup), and a complete ML pipeline that trains and evaluates two regression models. The Random Forest was retained as the best model with RMSE = 44,554, MAE = 1,738, and $R^2 = 0.1041$.

6.2 Improvement Proposals

Proposal 1 — Enriching the feature set with temporal and meteorological variables: the most impactful improvement would be to add features derived from the timestamp (hour of day, day of week, month, weekend indicator, season) and to join the weather data into the model training step (outdoor temperature, wind speed). Studies on similar datasets consistently show that these additions can push R^2 above 0.6 or 0.7, compared to the 0.10 achieved here. The Spark pipeline architecture we built is already designed to accommodate these additions easily.

Proposal 2 — Training on the full dataset with hyperparameter optimization: training on the full 20-million-row dataset rather than the 50,000-row sample would give the model access to the complete range of consumption patterns across all three years and all 16 sites. Pairing this with cross-validated hyperparameter tuning for the Random Forest (number of trees, maximum depth, minimum samples per leaf) and possibly exploring gradient boosting models such as XGBoost would likely yield significant metric improvements. Automating the periodic re-training with Apache Airflow would then make the platform fully operational in a production setting.

7. Annexes

AnnexA—Technical Architecture

- Storagelayer:HDFS—rawCSV files and Hive warehouse
- Warehouse layer: Apache Hive 3.1 — SQL-on-Hadoop with Parquet storage
- Processing layer: Apache Spark 2.3.1 — in-memory distributed computation
- ML layer: Spark MLlib — Pipeline-based machine learning
- Interface layer: Apache Zeppelin — interactive notebook environment
- Platform: HDP 3.0.1 (Hortonworks Data Platform) deployed via Docker on MacBook Pro M1 Pro, 32 GB RAM

Annex B — Execution Instructions

- 1. Start the HDP 3.0Dockercontainer and launch HDFS, Hive Metastore, HiveServer2, and Zeppelin services
- 2. Download the ASHRAE dataset from kaggle.com/competitions/ashrae-energy-prediction/data
- 3. Upload the CSV files to HDFS: `hdfs dfs -put <file> /user/hive/raw/<directory>/`
- 4. Run `hive_scripts.sql` via beeline to create all tables, views, and the materialized view
- 5. Open the `BigData_Energy_Project` notebook in Zeppelin (`localhost:9995`) and run the 5 cells in order

Annex C — Submitted Files

- `hive_scripts.sql`— all HiveQL commands with inline comments
- `spark_pipeline.py` — full Spark + MLlib pipeline with comments
- `BigData_Energy_Project.json` — exported Zeppelin notebook
- `rapport_bigdata.pdf` — this report
- `slides_bigdata.pdf` — presentation slides